PROJECT REPORT

Newton–Raphson Method Using Python:
A Root-Finding and Convergence Analysis Tool

Calculus, Exercise 4.6 (14th Edition)

1. Problem Statement

Many equations in science and engineering have the form $f(x) = 0$, where f is a nonlinear function. The values of x that satisfy the equation are called **roots**. For simple equations, such as a linear or a quadratic, a root can be found with an exact formula. For many other equations, for example those that mix polynomials, exponentials and trigonometric functions, no exact formula exists, and the root cannot be written down by algebra alone.

Numerical methods solve this difficulty. Instead of looking for an exact answer, a numerical method produces a sequence of better and better approximations and stops when the answer is accurate enough. The **Newton–Raphson method** is one of the most widely used of these methods because it uses the derivative of the function to make very efficient improvements.

Doing the repeated calculations by hand is slow and easy to get wrong. A Python program can repeat the same steps many times, keep a record of every step, and draw graphs that show how the approximations approach the root. This project builds such a program and applies it to the example equation $x^2 - 2 = 0$, whose positive root is $\sqrt{2}$.

2. Objectives

The objectives of this project are:

- To understand the mathematical idea behind the Newton–Raphson method.
- To implement the iterative formula in Python in a clear, readable program.
- To let the user set the function, the initial guess, the tolerance and the maximum number of iterations.
- To compute the derivative automatically with SymPy and to repeat the iteration until the tolerance is satisfied.
- To store every iteration in a Pandas table.
- To draw graphs of the function and of the convergence using Matplotlib.
- To evaluate the accuracy and speed of convergence, and to discuss the limitations of the method using the results the program actually produced.

3. Introduction and Mathematical Theory

The idea in simple words

Suppose we want to find where a curve $y = f(x)$ crosses the x-axis, and we have a first guess x_0. The curve may be bending, but near our guess it looks almost like a straight line. That straight line is the **tangent line** to the curve at the guess. We find where this tangent line crosses the x-axis, and we use that crossing point as our next, better guess. We then repeat the idea from the new point. Each repetition is called an **iteration**.

The formula

The tangent line at the point $(x_n, f(x_n))$ has slope $f'(x_n)$. Setting the height of this line to zero and solving for x gives the Newton–Raphson formula:

$$x_{n+1} = x_n - f(x_n) / f'(x_n) \qquad (1)$$

The symbols mean:

- x_n is the current approximation (the guess at step n). The first guess is x_0, called the **initial guess**.
- x_{n+1} is the next, improved approximation.
- $f(x_n)$ is the value of the function at the current approximation. If it is zero, we have found a root.
- $f'(x_n)$ is the derivative of the function at the current approximation. It is the slope of the tangent line and tells us how steeply the curve is rising or falling. It must **not** be zero, because the formula divides by it.

Stopping criterion and error

The method could continue forever, so we need a rule to stop. In this project the program measures the **absolute error** as the size of the most recent step:

$$E_n = | x_{n+1} - x_n | \qquad (2)$$

The iteration stops when E_n is smaller than a chosen **tolerance** (here 10^{-10}). A maximum number of iterations is also set as a safety limit, so that the program cannot run endlessly if the method fails.

Example function

Because no specific function was supplied for this report, the example equation is:

$$f(x) = x^2 - 2, \qquad f'(x) = 2x$$

The positive root of this equation is $\sqrt{2} \approx 1.41421356$. Substituting into Equation (1) gives the iteration formula for this example:

$$x_{n+1} = x_n - (x_n^2 - 2) / (2x_n) \qquad (3)$$

First iterations calculated by hand

Take the initial guess $x_0 = 2$.

Iteration 1: $f(2) = 2^2 - 2 = 2$ and $f'(2) = 4$, so $x_1 = 2 - 2/4 = 1.5$.

Iteration 2: $f(1.5) = 2.25 - 2 = 0.25$ and $f'(1.5) = 3$, so $x_2 = 1.5 - 0.25/3 = 1.416666\ldots$.

Iteration 3: $f(1.416667) \approx 0.0069444$ and $f'(1.416667) \approx 2.833333$, so $x_3 \approx 1.416667 - 0.0069444/2.833333 \approx 1.414216$.

These hand calculations agree with the first rows of the program output in Table 1.

4. Methodology and Algorithm

The work followed these stages: (1) choose the example function and study the theory; (2) write the Python program; (3) run the program and save the iteration table; (4) draw the graphs; (5) run extra tests with different starting guesses and invalid inputs; and (6) write the report using only the outputs that were produced.

Input and output variables

Item	Symbol / name in code	Value used
Function	FUNCTION_TEXT	$x^2 - 2$
Initial guess	X0	2.0
Tolerance	TOLERANCE	10^{-10}
Maximum iterations	MAX_ITERATIONS	50
Zero-derivative limit	DERIVATIVE_LIMIT	10^{-12}
Outputs	—	Iteration table, approximate root, final error, convergence status, graphs

Algorithm (pseudocode)

1. Read $f(x)$, the initial guess x_0, the tolerance and the maximum number of iterations. Check that the inputs are valid.
2. Use SymPy to find $f'(x)$.
3. Set $n = 1$.
4. Compute $f(x_n)$ and $f'(x_n)$. If either value is not finite, or $|f'(x_n)|$ is smaller than the derivative limit, stop and report the problem.
5. Compute $x_{n+1} = x_n - f(x_n) / f'(x_n)$ and the error $E_n = |x_{n+1} - x_n|$.
6. Store n, x_n, $f(x_n)$, $f'(x_n)$, x_{n+1} and E_n as a row of the table.
7. If E_n is smaller than the tolerance, stop: the method has converged.
8. If n has reached the maximum number of iterations, stop: the method has not converged. Otherwise set $n = n + 1$, replace x_n by x_{n+1}, and go back to step 4.

Flowchart

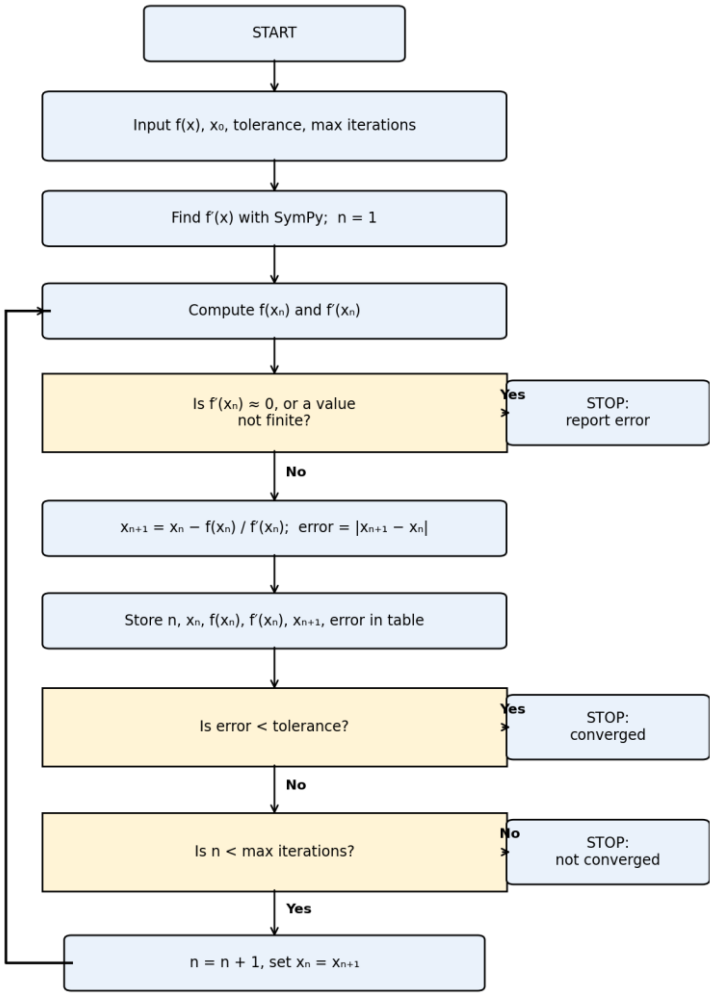


Figure 1. Flowchart of the Newton–Raphson program.

5. Python Implementation

How the program works

The program has four parts. **Part 1 (settings)** holds the function, the initial guess, the tolerance and the iteration limit in one place, so a beginner can change them easily. **Part 2 (build_functions)** converts the text of the function into Python functions and uses SymPy to differentiate it. The function text is written by the student inside the code, which avoids running arbitrary text typed by an unknown user. **Part 3 (newton_raphson)** is the main loop: it applies Equation (1), stores each row, and stops when the tolerance is met. It also protects against a zero derivative, non-finite numbers, invalid inputs, and failure to converge within the iteration limit. **Part 4 (make_graphs and main)** prints the results and saves the graphs.

Complete source code

```
"""
Newton-Raphson Method: Root Finding and Convergence Analysis
Calculus project (Exercise 4.6) - beginner friendly version.

Libraries used:
  NumPy      - numbers and graph values
  Pandas     - iteration table
  Matplotlib - graphs
  SymPy      - symbolic derivative f'(x)
"""


import math
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")              # lets the program save graphs without opening a window
import matplotlib.pyplot as plt
import sympy as sp

# ---------------------------------------------------------------------
# 1. SETTINGS (change these to try other problems)
# ---------------------------------------------------------------------
FUNCTION_TEXT = "x**2 - 2"    # f(x), written in Python style (x**2 means x squared)
X0 = 2.0                      # initial guess
TOLERANCE = 1e-10             # stop when |x_new - x_old| < TOLERANCE
MAX_ITERATIONS = 50           # safety limit
DERIVATIVE_LIMIT = 1e-12      # if |f'(x)| is smaller than this, we treat it as zero



# ---------------------------------------------------------------------
# 2. BUILD f(x) AND f'(x)
# ---------------------------------------------------------------------
def build_functions(function_text):
    """Turn the text into Python functions for f(x) and f'(x) using SymPy."""
    x = sp.symbols("x")
    try:
        f_expr = sp.sympify(function_text, locals={"x": x})
    except (sp.SympifyError, SyntaxError) as error:
        raise ValueError(f"Could not read the function '{function_text}'.") from error
    df_expr = sp.diff(f_expr, x)                      # symbolic derivative
    f = sp.lambdify(x, f_expr, "numpy")               # fast numeric versions
    df = sp.lambdify(x, df_expr, "numpy")
    return f, df, f_expr, df_expr



# ---------------------------------------------------------------------
# 3. NEWTON-RAPHSON LOOP
# ---------------------------------------------------------------------
def newton_raphson(f, df, x0, tol, max_iter):
    """Return (table, root, converged, message)."""
    # Check the inputs first
    if tol <= 0 or max_iter < 1 or not math.isfinite(x0):
        raise ValueError("Tolerance must be positive, max_iterations at least 1, "
                         "and the initial guess must be a finite number.")

    rows = []
    x_old = float(x0)
    converged = False
    message = "Maximum iterations reached without convergence."

    for n in range(1, max_iter + 1):
        fx = float(f(x_old))
        dfx = float(df(x_old))

        if not (math.isfinite(fx) and math.isfinite(dfx)):
            message = f"Stopped: non-finite value at iteration {n}."
            break
        if abs(dfx) < DERIVATIVE_LIMIT:
            message = f"Stopped: derivative is (almost) zero at x = {x_old}."
            break

        x_new = x_old - fx / dfx                      # Newton-Raphson formula
        error = abs(x_new - x_old)                    # size of the last step
        rows.append({"Iteration": n, "x_n": x_old, "f(x_n)": fx,
                     "f'(x_n)": dfx, "x_(n+1)": x_new, "Abs Error": error})
```

```
        if not math.isfinite(x_new):
            message = f"Stopped: non-finite value at iteration {n}."
            break
        x_old = x_new
        if error < tol:
            converged = True
            message = "Converged: step size is smaller than the tolerance."
            break

    return pd.DataFrame(rows), x_old, converged, message



# ---------------------------------------------------------------------
# 4. GRAPHS
# ---------------------------------------------------------------------
def make_graphs(f, table, root, x0, prefix="fig"):
    # Graph 1: the function with the root marked
    xs = np.linspace(-0.5, x0 + 0.5, 400)
    plt.figure(figsize=(7, 4.5))
    plt.plot(xs, f(xs), label="f(x) = x² − 2", color="tab:blue")
    plt.axhline(0, color="black", linewidth=0.8)
    plt.scatter([root], [0], color="red", zorder=5, label=f"Approximate root ≈ {root:.6f}")
    plt.annotate("root", xy=(root, 0), xytext=(root - 0.9, 1.2),
                 arrowprops=dict(arrowstyle="->"))
    plt.title("Graph of f(x) = x² − 2 and Its Positive Root")
    plt.xlabel("x"); plt.ylabel("f(x)")
    plt.grid(True, alpha=0.3); plt.legend()
    plt.tight_layout(); plt.savefig(f"{prefix}1_function.png", dpi=200); plt.close()

    # Graph 2: approximations against iteration number
    iters = [0] + list(table["Iteration"])
    approx = [x0] + list(table["x (n+1)"])
    plt.figure(figsize=(7, 4.5))
    plt.plot(iters, approx, "o-", color="tab:green", label="Approximation x n")
    plt.axhline(math.sqrt(2), color="red", linestyle="--", label="√2 (reference)")
    plt.title("Convergence of the Approximations")
    plt.xlabel("Iteration number n"); plt.ylabel("Approximation x_n")
    plt.grid(True, alpha=0.3); plt.legend()
    plt.tight layout(); plt.savefig(f"{prefix}2 convergence.png", dpi=200); plt.close()

    # Graph 3: absolute error (log scale, zeros replaced to avoid log(0))
    err = np.array(table["Abs Error"], dtype=float)
    err safe = np.where(err > 0, err, np.finfo(float).tiny)
    plt.figure(figsize=(7, 4.5))
    plt.semilogy(table["Iteration"], err_safe, "s-", color="tab:purple", label="|x_(n+1) −
x n|")
    plt.axhline(TOLERANCE, color="red", linestyle="--", label=f"Tolerance = {TOLERANCE:g}")
    plt.title("Error Convergence (Logarithmic Scale)")
    plt.xlabel("Iteration number n"); plt.ylabel("Absolute error")
    plt.grid(True, which="both", alpha=0.3); plt.legend()
    plt.tight_layout(); plt.savefig(f"{prefix}3_error.png", dpi=200); plt.close()



# ---------------------------------------------------------------------
# 5. MAIN PROGRAM
# ---------------------------------------------------------------------
def main():
    f, df, f expr, df expr = build functions(FUNCTION TEXT)
    print(f"f(x)  = {f expr}")
    print(f"f'(x) = {df expr}  (found by SymPy)")
    print(f"Initial guess = {X0}, tolerance = {TOLERANCE}, max iterations =
{MAX_ITERATIONS}\n")

    table, root, converged, message = newton_raphson(f, df, X0, TOLERANCE, MAX_ITERATIONS)

    pd.set_option("display.float_format", lambda v: f"{v:.12g}")
    pd.set_option("display.width", 140)
    print(table.to_string(index=False))
    table.to csv("iteration table.csv", index=False)

    print("\n--- Final results ---")
    print(f"Approximate root : {root:.15f}")
    print(f"f(root)          : {float(f(root)):.3e}")
    print(f"Iterations used  : {len(table)}")
    print(f"Final error      : {table['Abs Error'].iloc[-1]:.3e}")
    print(f"Tolerance        : {TOLERANCE:g}")
    print(f"Converged        : {converged}  ({message})")
```

```
    print(f"True sqrt(2)     : {math.sqrt(2):.15f}")
    print(f"Actual error vs sqrt(2): {abs(root - math.sqrt(2)):.3e}")


    make_graphs(f, table, root, X0)


    # ---- Extra experiments used in the Discussion section ----
    print("\n--- Different starting guesses (same f, same tolerance) ---")
    for guess in [1.0, 1.5, 2.0, 10.0, -3.0, 100.0]:
        t, r, c, m = newton_raphson(f, df, guess, TOLERANCE, MAX_ITERATIONS)
        print(f"x0 = {guess:>6}: iterations = {len(t):>2}, root = {r:.12f}, converged = {c}")


    print("\n--- Derivative-zero test (x0 = 0) ---")
    t, r, c, m = newton_raphson(f, df, 0.0, TOLERANCE, MAX_ITERATIONS)
    print(f"Converged = {c}. Message: {m}")


    print("\n--- Invalid input test (tolerance = -1) ---")
    try:
        newton_raphson(f, df, 1.0, -1, 10)
    except ValueError as error:
        print("ValueError caught:", error)



if __name__ == "__main__":
    main()
```

6. Results and Calculations

All values in this section were produced by running the program above (Python 3.12.3). Nothing in this section was estimated or typed in by hand.

Iteration table

The settings were: $f(x) = x^2 - 2$, $f'(x) = 2x$, initial guess $x_0 = 2$, tolerance 10^{-10}, maximum 50 iterations.

n	x_n	$f(x_n)$	$f'(x_n)$	x_{n+1}	$E_n = \lvert x_{n+1} - x_n \rvert$
1	2.000000000000	2	4	1.500000000000	5.0000e-01
2	1.500000000000	0.25	3	1.416666666667	8.3333e-02
3	1.416666666667	0.00694444	2.833333333	1.414215686275	2.4510e-03
4	1.414215686275	6.0073e-06	2.828431373	1.414213562375	2.1239e-06
5	1.414213562375	4.51061e-12	2.828427125	1.414213562373	1.5947e-12

Final results

Quantity	Computed value
Initial guess	2.0
Approximate root	1.414213562373095
f(root)	4.441×10^{-16}
Number of iterations	5
Final error (last step size)	1.595×10^{-12}
Tolerance	1×10^{-10}
Converged?	Yes: the step size is smaller than the tolerance
Difference from √2 (computer value)	0 (the two double-precision numbers are identical)

The tiny value $f(\text{root}) = 4.441 \times 10^{-16}$ is not exactly zero because computers store decimal numbers with limited precision, so the last digits contain small rounding effects.

Convergence check using high-precision arithmetic

Standard computer numbers stop showing real error after about 16 digits. To see how the true error behaves, a second small program (*verify_convergence.py*, supplied with this report) repeated the same five iterations using 50-digit decimal arithmetic and compared each approximation with the true value of √2. The ratio in the last column is e_n / e_{n-1}^2, where $e_n = |x_n - \sqrt{2}|$ is the true error.

n	x_n	True error e_n	e_n / e_{n-1}^2
0	2.000000000000000	5.857864×10^{-1}	—
1	1.500000000000000	8.578644×10^{-2}	0.25000
2	1.416666666666667	2.453104×10^{-3}	0.33333
3	1.414215686274510	2.123901×10^{-6}	0.35294
4	1.414213562374690	1.594862×10^{-12}	0.35355
5	1.414213562373095	8.992928×10^{-25}	0.35355

For this example the theoretical value of $f''(r) / (2f'(r))$ at the root $r = \sqrt{2}$ is $1/(2\sqrt{2}) = 0.35355$, which matches the last two ratios in the table.

Effect of the initial guess

The same program was run with other starting guesses, using the same function and tolerance.

Initial guess x_0	Iterations	Final approximation	Converged?
1	5	1.414213562373	Yes
1.5	4	1.414213562373	Yes
2	5	1.414213562373	Yes
10	8	1.414213562373	Yes
-3	6	-1.414213562373	Yes
100	11	1.414213562373	Yes
0	0	—	No (derivative zero)

When the initial guess was 0, the program stopped at once with the message “Stopped: derivative is (almost) zero at x = 0.0”. A test with an invalid tolerance of −1 was also rejected correctly with a ValueError message.

7. Graphs and Data Visualization

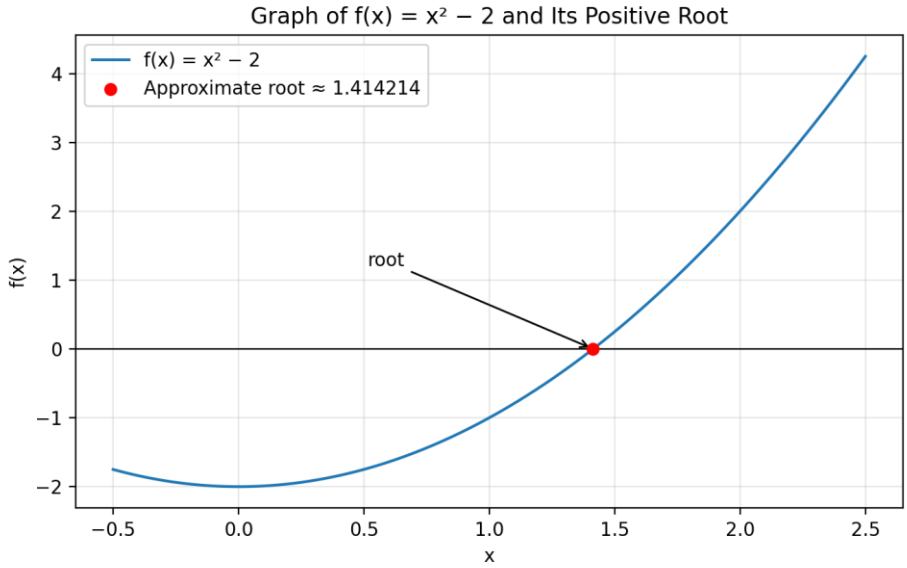


Figure 2 shows that the curve crosses the x-axis at about $x = 1.4142$, where the red point marks the root found by the program. The curve is steep and smooth near the root, which is a favourable situation for the method.

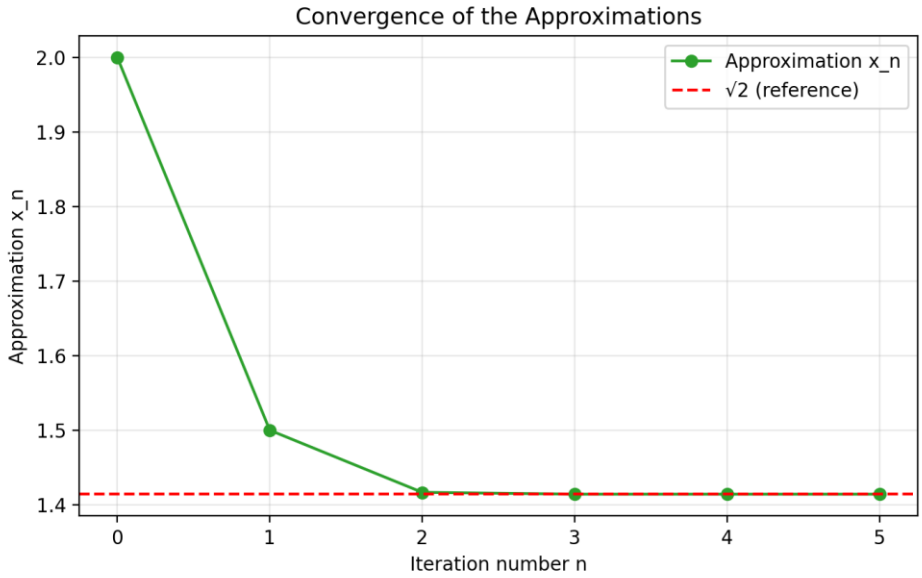


Figure 3 starts at the initial guess 2 and shows the approximations falling quickly onto the dashed line at √2. After about three iterations the points are already too close to the line to be distinguished by eye.

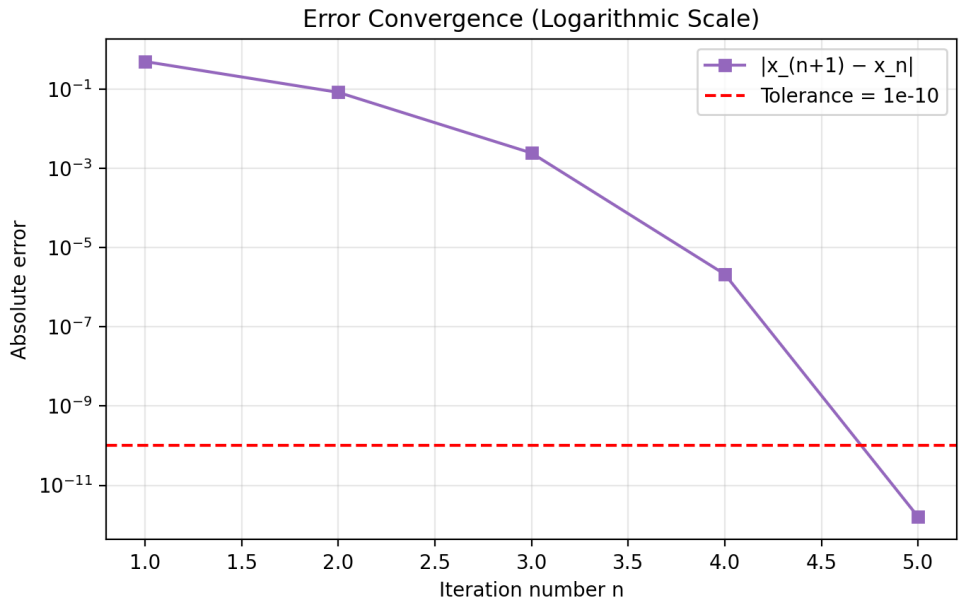


Figure 4 uses a logarithmic vertical axis because the error shrinks from about 10^{-1} to about 10^{-12}. The line bends downward more and more steeply, which means the error falls faster at every step. The curve crosses the red tolerance line between iterations 4 and 5, which is why the program stopped at iteration 5.

8. Discussion and Findings

Speed of convergence. The program reached the tolerance of 10^{-10} in only 5 iterations from the initial guess 2. The step sizes in Table 1 were 0.5, 0.0833, 0.00245, 2.12×10^{-6} and 1.59×10^{-12}.

How the error changed. The true errors in Table 3 went from 8.6×10^{-2} to 2.5×10^{-3}, 2.1×10^{-6} and 1.6×10^{-12}. Each new error is roughly a constant multiple (about 0.35) of the *square* of the previous error. This behaviour is called **quadratic convergence**, and in practice it means the number of correct digits roughly doubles with each iteration once the guess is close to the root.

Accuracy. The final approximation 1.414213562373095 equals the computer's value of √2 to all digits that double-precision numbers can show, and f(root) is about 4.4×10^{-16}. The tolerance of 10^{-10} was therefore met with a large margin.

Effect of the initial guess. For the six non-zero guesses tested, the method converged each time (Table 4). Guesses closer to the root needed fewer iterations: 4 iterations for 1.5, but 8 for 10 and 11 for 100. A positive guess gave +√2 and the negative guess −3 gave −√2, so the sign of the starting point decided which root was found. These observations apply to this example only; they do not prove that every starting guess works for every function.

Zero derivative. The derivative $2x$ is zero at $x = 0$. Because the formula divides by $f'(x_n)$, the first step is impossible, and the program correctly stopped with a clear message instead of crashing or producing an infinite value.

Advantages. In this project the method was fast (5 iterations), needed only one starting value, and was easy to program. Using SymPy removed the need to differentiate by hand.

Disadvantages seen or expected. The method needs the derivative and fails when it is zero, as the test at $x_0 = 0$ showed. It also needs a reasonable starting point; a guess far from the root took more iterations (11 for 100).

9 Limitations and Conclusion

Limitations

- The results are for one example function, $f(x) = x^2 - 2$. Other functions can behave differently, and no claim is made about them from these results.
- The stopping rule uses the size of the last step, not the exact error, which is not known in a real problem. The step size is a good indicator here, but it is not a guarantee for every function.
- The method needs the derivative to be non-zero near each iterate. For functions with a derivative equal to or close to zero, or with several roots close together, a poor starting guess may fail or jump to a different root. Only the zero-derivative case at $x_0 = 0$ was tested in this project.
- The program finds one root per run, and the answer depends on the initial guess.
- Computer arithmetic has limited precision, so the final digits of any result contain small rounding errors.
- The function in this report was chosen as an example because the exact Exercise 4.6 question was not supplied. To answer the assigned exercise, change FUNCTION_TEXT and X0 at the top of the program and run it again.

Conclusion

This project explained the Newton–Raphson method, implemented it in Python with NumPy, Pandas, Matplotlib and SymPy, and tested it on $f(x) = x^2 - 2$. Starting from $x_0 = 2$, the program found the root 1.414213562373095 in 5 iterations with a final step size of 1.6×10^{-12}, well below the tolerance of 10^{-10}. A high-precision check showed that the true error was roughly squared at each step, which is the signature of quadratic convergence. Further tests showed that other starting guesses also converged for this function, and that the program handled a zero derivative and an invalid tolerance safely. The tables and graphs made the speed of convergence easy to see, which shows the value of visualization in numerical work.

10. References

The references below use APA style. The links could not be opened while this report was being prepared because the working environment had no internet access, so please check each link once before submitting.

Burden, R. L., Faires, J. D., & Burden, A. M. (2016). *Numerical analysis* (10th ed.). Cengage Learning.

Hass, J., Heil, C., & Weir, M. D. (2018). *Thomas' calculus* (14th ed.). Pearson. [Replace with the exact textbook used for Exercise 4.6, if different.]

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Meurer, A., Smith, C. P., Paprocki, M., et al. (2017). SymPy: Symbolic computing in Python. *PeerJ Computer Science, 3*, e103. https://doi.org/10.7717/peerj-cs.103

Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3/

Weisstein, E. W. (n.d.). Newton's method. *MathWorld—A Wolfram Web Resource*. https://mathworld.wolfram.com/NewtonsMethod.html